

靜態檔案的設定

```
# Static files (CSS, JavaScript, Images)
# 靜態檔案的目錄，這裡是放置靜態檔案的目錄，Django會自動去這個目錄尋找靜態檔案
STATIC_DIR = os.path.join(BASE_DIR, 'static') # 置放靜態檔案的目錄 Directory where
static files are collected

# 生產或佈署階段 nginx須設定遇到URL前綴去STATIC_ROOT設定的目錄下去尋找靜態檔案
# STATIC_URL是靜態檔案的URL前綴，當瀏覽器請求靜態檔案時，會使用這個URL前綴
# Django runserver不會理會STATIC_URL的設定，只會取URL前綴之後的路徑的靜態檔案
STATIC_URL = '/static/poa/' # URL prefix for static files

# 生產或佈署階段
# 這裡是靜態檔案的收集目錄，當執行python manage.py collectstatic時，會去
STATICFILES_DIRS定義的目錄去收集靜態檔案複製到這個目錄下
# nginx會去這個目錄下尋找靜態檔案
# 這裡是docker-compose的時候使用的靜態檔案設定
STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles/poa/')

# 生產或佈署階段
# 蒐集collectstatic時，去這裡定義的目錄蒐集靜態static檔案
# 若有新的檔案，需要重新製作新的容器，才會再去更新python manage.py collectstaticfiles
STATICFILES_DIRS = [
    os.path.join(BASE_DIR, "static"),
    # BASE_DIR / 'static', # 這種寫法也可以
    # STATIC_DIR, # 這種寫法也可以
]
```

以下為各設定項目用途、開發與生產階段的說明，以及 Django server 與 nginx 的使用方式：

1. `STATIC_URL = '/static/poa/'`

- **用途：**設定靜態檔案在網頁上的 URL 前綴 (prefix) 。
 - **開發階段：**Django 開發伺服器 (runserver) 會自動處理這個 URL，將 `/static/poa/xxx` 的請求對應到本機的靜態檔案目錄。
 - **生產階段：**nginx 會根據這個 URL 前綴，將請求導向實際的靜態檔案目錄 (通常是 `STATIC_ROOT`) 。
-

2. `STATIC_DIR = os.path.join(BASE_DIR, 'static')`

- **用途：**專案原始靜態檔案的存放目錄 (開發時放置 CSS、JS、圖片等) 。
 - **開發階段：**Django 會直接從這個目錄讀取靜態檔案 (透過 `STATICFILES_DIRS`) 。
 - **生產階段：**不直接被 nginx 使用，但會被 `collectstatic` 指令收集到 `STATIC_ROOT`。
-

3. `STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles/poa/')`

- 用途：collectstatic 指令會將所有靜態檔案收集到這個目錄，供生產環境（nginx）使用。
- 開發階段：通常不會用到這個目錄。
- 生產階段：nginx 會設定 root 指向這個目錄，所有 `/static/poa/` 的請求都會從這裡讀取檔案。

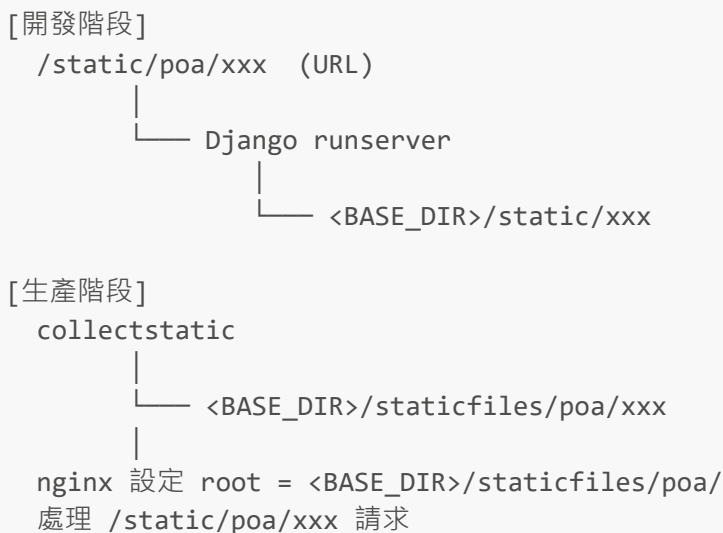
4. `STATICFILES_DIRS = [os.path.join(BASE_DIR, "static")]`

- 用途：告訴 Django 額外要去哪裡找靜態檔案（除了 `app/static` 以外）。
- 開發階段：Django runserver 會從這些目錄直接提供靜態檔案。
- 生產階段：collectstatic 會從這些目錄收集檔案到 `STATIC_ROOT`。

開發與生產流程總結

- 開發階段（Django runserver）
 - 直接從 `STATICFILES_DIRS` 指定的目錄（如 `static`）讀取檔案。
 - 網頁請求 `/static/poa/xxx`，Django 會去 `static/xxx` 找檔案。
- 生產階段（nginx）
 - 先執行 `python manage.py collectstatic`，將所有靜態檔案收集到 `STATIC_ROOT`。
 - nginx 設定 root 指向 `STATIC_ROOT`，處理 `/static/poa/` 的請求。

圖解



重點：

- 開發用 `STATICFILES_DIRS`，生產用 `STATIC_ROOT`。
- `STATIC_URL` 決定網址前綴，目錄結構不需有 `poa` 子目錄。
- nginx 只會讀取 `STATIC_ROOT`，不會讀取 `STATICFILES_DIRS`。

Django server 讀取靜態檔案

Django server 讀取靜態檔案說明如下：

- `STATIC_URL = '/static/poa/'` 只影響「網址」路徑，不影響實體目錄結構。
- `STATICFILES_DIRS = [os.path.join(BASE_DIR, "static")]` 告訴 Django 去 `<BASE_DIR>/static` 這個資料夾找靜態檔案。
- Django runserver 會自動把 `/static/poa/xxx` 的網址對應到 `<BASE_DIR>/static/xxx` 的檔案。

例子

- 你有檔案：`<BASE_DIR>/static/assets_api_intronduction/logo.png`
- 你在 HTML 用：``
- Django 會去 `<BASE_DIR>/static/assets_api_intronduction/logo.png` 找這個檔案

你不需要在 `static` 目錄下建立 `poa` 子目錄。

結論

- 只要你的檔案在 `<BASE_DIR>/static/assets_api_intronduction/...`，網址用 `/static/poa/assets_api_intronduction/...`，runserver 就能正確抓到。
 - Django 會自動把 `/static/poa/` 這個 prefix 去掉，剩下的路徑去 `STATICFILES_DIRS` 指定的目錄找檔案。
-

如需測試，可直接用 runserver，網址 `/static/poa/assets_api_intronduction/xxx` 應該可以正確顯示檔案。

static 和 staticfiles 兩個概念

在標準的 Django 部署流程中，`static` 和 `staticfiles` 這兩個概念（以及對應的目錄）通常都有其必要性，它們扮演不同的角色：

1. `static` 目錄 (由 `STATICFILES_DIRS` 指定):

- **用途:** 這是您在開發過程中放置專案層級靜態檔案（如 CSS, JavaScript, 圖片）的來源目錄。Django 的開發伺服器 (runserver 且 `DEBUG=True`) 會從這裡以及各個 app 內的 `static` 子目錄尋找並提供靜態檔案。
- **內容:** 只包含您自己為這個專案建立的靜態檔案。

2. `staticfiles` 目錄 (由 `STATIC_ROOT` 指定):

- **用途:** 這是執行 `python manage.py collectstatic` 指令時，所有靜態檔案會被收集彙整到的目的地目錄。這個指令會尋找所有 `STATICFILES_DIRS` 指定的路徑以及所有 `INSTALLED_APPS` 中 app 的 `static` 子目錄，將找到的檔案複製到 `STATIC_ROOT` 指定的這個目錄下。在生產環境中，您的網頁伺服器（如 Nginx）會被設定成直接從這個 `staticfiles` 目錄提供靜態檔案服務，而不是透過 Django。
- **內容:** 在執行 `collectstatic` 之後，這個目錄會包含：
 - 您專案 `static` 目錄的所有內容。

- 所有已安裝 app (包括 Django admin 等內建 app) 的 `static` 子目錄中的所有內容。
- 因此，它的內容會比您專案的 `static` 目錄更完整，是整個專案運行所需的所有靜態檔案的集合。

結論：

- **是否有必要?** 是的，為了標準的開發與部署流程，區分來源 (`static`) 和部署用的集合目錄 (`staticfiles`) 是必要的。
- **內容是否一樣?** 不一樣。`staticfiles` 是執行 `collectstatic` 後的結果，它包含了您專案的 `static` 檔案以及所有 app 的靜態檔案。在執行 `collectstatic` 之前，`staticfiles` 目錄可能不存在或是空的。

在您的 Docker 設定中，`collectstatic` 指令會在 `web` 容器內執行，將所有靜態檔案收集到 `/app/staticfiles` (對應到 `static_volume`)，然後 `nginx` 容器會掛載同一個 `static_volume` 並從中提供靜態檔案服務。這個流程正是利用了 `STATIC_ROOT` 的設計目的。

Django 靜態檔案設定說明

以下是這四個設定的整理說明：

STATIC_URL

- **用途：**定義在 HTML 模板中 `{% static %}` 標籤產生的靜態檔案網址開頭。
- **誰在用：**Django (HTML 模板)、Nginx (反向代理時也會用到這個路徑)。
- **階段：**開發 + 生產皆會用到。

STATIC_DIR

- **用途：**通常是你自己定義的變數，指向專案內部存放靜態檔案的資料夾 (如 `static/`)，方便後續設定。
- **誰在用：**Django 設定檔內部用來組合路徑，實際上 Django 不會直接用這個變數。
- **階段：**開發 + 生產皆可用 (只是輔助變數)。

STATIC_ROOT

- **用途：**指定 `python manage.py collectstatic` 時，所有靜態檔案要被「集中複製」到哪個資料夾。
- **誰在用：**Nginx 或 Web Server (生產環境時直接從這裡提供靜態檔案)。
- **階段：**生產階段才會用到 (Django `collectstatic` 時)。

STATICFILES_DIRS

- **用途：**告訴 Django 開發階段要去哪裡找你自己放的靜態檔案 (通常是 `static/` 資料夾)。
- **誰在用：**Django 的開發伺服器 (`runserver`)。
- **階段：**開發階段才會用到。

總結表格

設定項目	用途說明	誰會用到？	何時用？
STATIC_URL	HTML 裡 <code>{% static %}</code> 產生的網址開頭	Django (HTML) 、 Nginx	開發 + 生產
STATIC_DIR	你自訂的靜態檔案資料夾路徑變數	只在 settings.py 內部用	開發 + 生產
STATIC_ROOT	collectstatic 集中複製靜態檔案的目的地	Nginx/Web Server	生產
STATICFILES_DIRS	開發階段 Django 去哪找你自己放的靜態檔案	Django 開發伺服器	開發

簡單記憶法：

- 開發時 Django 直接從 `STATICFILES_DIRS` 指定的資料夾找檔案。
- 生產時要先 collectstatic 到 `STATIC_ROOT`，Nginx 再從這裡提供檔案。
- `STATIC_URL` 是網址開頭，`STATIC_DIR` 只是你方便使用的變數。

llm專案

在llm專案中，這兩個設定用來處理靜態檔案的提供方式：

你的設定方式是正確的，說明如下：

- `STATIC_URL = '/static/llm/'`
這會讓你在 HTML 裡用 `{% static 'xxx.js' %}` 時，產生 `/static/llm/xxx.js` 這樣的網址，這樣 Nginx 可以根據這個路徑提供靜態檔案。
- `STATIC_ROOT = os.path.join(BASE_DIR, 'staticfiles/llm')`
這是 collectstatic 時，所有靜態檔案會被集中複製到的資料夾。生產環境下，Nginx 會直接從這裡提供檔案。
- `STATICFILES_DIRS = [os.path.join(BASE_DIR, "static")]`
這讓 Django 開發伺服器在開發階段能直接從 `static/` 目錄載入你的靜態檔案。
- `STATIC_DIR` 自己的靜態檔案（如 CSS、JS、圖片等）放在專案根目錄下的 `static/` 目錄裡。
`STATIC_DIR = os.path.join(BASE_DIR, 'static')` 這行只是方便你在設定檔中引用 `static` 目錄的路徑。
你可以把自己的靜態檔案（如 CSS、JS、圖片等）放在專案根目錄下的 `static/` 目錄裡，Django 會根據 `STATICFILES_DIRS` 去這個目錄找檔案。

注意事項：

- 生產環境時，請確保 Nginx 的設定有對應 `/static/llm/` 指向 `staticfiles/llm` 目錄。
- `STATIC_ROOT` 目錄不要手動放檔案，所有檔案都應該放在 `static/` 或 app 的 `static` 目錄，然後用 `collectstatic` 集中到 `STATIC_ROOT`。
- 這樣設定可以避免和其他專案的靜態檔案衝突。

只要 `STATICFILES_DIRS` 有包含這個路徑，開發階段 Django 就能正確載入這些檔案。生產階段則會用 `collectstatic` 把這些檔案集中到 `STATIC_ROOT`。

結論：

你的設定符合 Django 與 Nginx 的最佳實踐，開發與生產階段都能正確運作。

靜態檔案置放於app目錄底下的做法

將靜態檔案放在 `app_llm/static/` 目錄下是 Django 官方推薦的做法之一，這樣做**沒有問題**，而且有其特定用途和好處：

什麼情況要放在 app 目錄底下？

- 當某些靜態檔案只屬於**特定 app**（例如：該 app 專用的 JS、CSS、圖片等），建議放在該 app 的 `static/` 目錄下，例如 `app_llm/static/`。
- 這樣可以讓不同 app 的靜態檔案**互不干擾**、方便管理。

好處

1. **模組化管理**：每個 app 的靜態檔案都放在自己的資料夾，方便維護與移植。
2. **避免命名衝突**：不同 app 可以有同名的靜態檔案（如 `main.js`），Django `collectstatic` 會自動處理合併。
3. **可重複使用**：如果你要把某個 app 拿到別的專案用，靜態檔案也會一起帶走，不會漏掉。

注意事項

- 如果你有設定 `STATICFILES_DIRS`，Django 會同時搜尋根目錄的 `static/` 和各 app 下的 `static/`。
- 生產環境時，記得執行 `python manage.py collectstatic`，Django 會把所有 app 的靜態檔案集中到 `STATIC_ROOT`。

總結

- 放在 `app_llm/static/` 沒問題，適合 app 專用靜態檔案。
- 放在專案根目錄的 `static/`，適合多個 app 共用的靜態檔案。
- 兩者可以同時存在，Django 都會處理。

不會發生名稱衝突，因為 Django `collectstatic` 會將 app 目錄下的 `static` 檔案自動加上 app 名稱作為子目錄。例如：

- `app_a/static/js/main.js` 會被收集到 `STATIC_ROOT/app_a/js/main.js`
- `app_b/static/js/main.js` 會被收集到 `STATIC_ROOT/app_b/js/main.js`

這樣即使不同 app 有同名檔案，也不會互相覆蓋。

只有當你在不同 app 的 `static` 目錄下放了**完全相同的路徑與檔名**（例如都放在 `static/js/main.js`，且沒有 app 名稱這層），才有可能發生衝突，但 Django 預設會以 app 名稱分層來避免這種情況。

建議：

- 在每個 app 的 static 目錄下建立以 app 名稱為子目錄的結構（如 `app_a/static/app_a/`），這是 Django 官方推薦的方式，可以完全避免衝突。
- 例如：`app_a/static/app_a/main.js`，`app_b/static/app_b/main.js`

這樣 `collectstatic` 之後會是：

- `STATIC_ROOT/app_a/main.js`
- `STATIC_ROOT/app_b/main.js`

總結：

只要遵循 app 名稱分層的慣例，不會有覆蓋或名稱衝突的問題。

根據你的說法：不會發生名稱衝突，因為 Django `collectstatic` 會將 app 目錄下的 static 檔案自動加上 app 名稱作為子目錄。例如：

`app_a/static/js/main.js` 會被收集到 `STATIC_ROOT/app_a/js/main.js` `app_b/static/js/main.js` 會被收集到 `STATIC_ROOT/app_b/js/main.js` 這樣即使不同 app 有同名檔案，也不會互相覆蓋。只有當你在不同 app 的 static 目錄下放了完全相同的路徑與檔名（例如都放在 `static/js/main.js`，且沒有 app 名稱這層），才有可能發生衝突，但 Django 預設會以 app 名稱分層來避免這種情況。

建議：

在每個 app 的 static 目錄下建立以 app 名稱為子目錄的結構（如 `app_a/static/app_a/`），這是 Django 官方推薦的方式，可以完全避免衝突。例如：`app_a/static/app_a/main.js`，`app_b/static/app_b/main.js` 這樣 `collectstatic` 之後會是：

`STATIC_ROOT/app_a/main.js` `STATIC_ROOT/app_b/main.js` 總結：只要遵循 app 名稱分層的慣例，不會有覆蓋或名稱衝突的問題。

`app_a`的html模板的讀取靜態檔案的寫法，與`app_b`html模板的讀取靜態檔案的寫法，請舉例。

以下是 `app_a` 和 `app_b` 的 HTML 模板讀取靜態檔案的寫法範例：

假設檔案結構如下：

- `app_a/static/app_a/main.js`
 - `app_b/static/app_b/main.js`
-

`app_a` 的 HTML 模板

```
{% load static %}
<script src="{% static 'app_a/main.js' %}"></script>
```

`app_b` 的 HTML 模板

```
{% load static %}
<script src="{% static 'app_b/main.js' %}"></script>
```

說明：

- `{% static 'app_a/main.js' %}` 會對應到 `collectstatic` 後的 `/static/llm/app_a/main.js`
- `{% static 'app_b/main.js' %}` 會對應到 `collectstatic` 後的 `/static/llm/app_b/main.js`

這樣就能確保不同 app 的靜態檔案互不干擾，且模板寫法清楚明確。